

Module 1 Cheatsheet — Python for Data Engineering

8 concepts · 1 analogy each · 1 mental formula each · 1 golden rule each. Memorize the story, not the syntax.

1 Virtual Environments (venv)

Analogy: Separate Kitchens. Italian kitchen and Japanese kitchen. Each has its own ingredients (packages). You never mix soy sauce into pasta — and project A's pandas version never breaks project B.

```
[ Your PC ] → [ Kitchen 1: project-A ]   pandas 2.2, numpy 1.26
              → [ Kitchen 2: project-B ]   pandas 1.5 (old, but isolated = safe)
```

```
# 1. Build the kitchen (once per project)
python -m venv .venv

# 2. Step into the kitchen (every work session)
.venv\Scripts\activate      # Windows   (Linux/Mac: source .venv/bin/activate)

# 3. Stock the shelves
pip install pandas numpy
```

Mental formula — 3 questions:

1. **BUILD** kitchen? → `python -m venv .venv`
2. **ENTER** kitchen? → `activate`
3. **STOCK** shelves? → `pip install ...`

Golden rule: *One project = one kitchen. Never cook in the global kitchen.*

2 Type Hints

Analogy: Recipe with Measurements. "Add flour" (untyped) vs "Add 200g flour" (typed). Same recipe — but only one can be checked before you bake. Type hints let tools catch mistakes *before* the code runs.

```
# Reading a hint:  name : type = default      and      -> return type
def clean_price(raw: str, currency: str = "EUR") -> float:
    return float(raw.replace(",", "."))

scores: list[float] = [7.5, 8.0]
lookup: dict[str, int] = {"NL": 31}
maybe: Optional[str] = None          # "str OR None" — must check before use!
```

Mental formula — read any signature in 3 steps:

1. **WHAT goes in?** → look after each `:`
2. **WHAT comes out?** → look after `->`
3. **CAN it be missing?** → `Optional[...]` means check for `None` first

Golden rule: *Untyped code is a recipe without measurements — it might work, but nobody can trust it.*

3 Dataclasses

Analogy: Pre-printed Form. Instead of hand-writing "Name: __, Price: __" on blank paper every time (writing `__init__`, `__repr__`, `__eq__` yourself), you grab a pre-printed form. `@dataclass` prints the form for you.

You write: the FIELDS (name + type + default)

Python writes: `__init__` `__repr__` `__eq__` (the boring 10+ lines)

```
from dataclasses import dataclass, field

@dataclass
class Product:
    name: str # required
    price: float = 0.0 # default value
    tags: list[str] = field(default_factory=list) # NEVER tags=[] (shared bug!)

    def with_vat(self) -> float: # methods work like normal classes
        return self.price * 1.21

p = Product("Chair", 100.0) # __init__ was auto-generated
```

Mental formula:

1. **STORING data?** → `@dataclass`, list the fields with types
2. **DEFAULT is a list/dict?** → `field(default_factory=list)`, never `=[]`
3. **COMPLEX behavior?** → then (and only then) a regular class

Golden rule: *If the class mostly holds data, it's a dataclass.*

4 NumPy

Analogy: Factory Conveyor Belt vs Hand Work. A Python loop picks up items one by one by hand. NumPy puts the whole array on a conveyor belt and processes everything in one machine pass — 50–100× faster. That belt is called **vectorization**.

```
import numpy as np
prices = np.array([100, 250, 80, 40])

prices * 1.21                                # whole belt at once – NO loop (vectorization)
prices[prices > 90]                          # boolean mask = quality-control gate → [100 250]
prices.mean(), prices.sum()                  # aggregation: belt → one number
matrix + row                                # broadcasting: small array auto-stretches to fit
```

Mental formula — before writing a loop, ask:

1. **MATH on all items?** → just write it: `arr * 2`
2. **FILTER items?** → boolean mask: `arr[arr > x]`
3. **ONE number from many?** → aggregate: `.sum()` `.mean()` `.std()`

Golden rule: *If you write for over numbers, stop — the conveyor belt (NumPy) does it faster.*

5 pandas

Analogy: Excel with a Robot Assistant. A DataFrame is the spreadsheet; instead of clicking, you tell the robot what to do. Every daily task maps to one Excel action you already know:

You want (Excel)	Robot command (pandas)
Open a file	<code>df = pd.read_csv("data/barbie.csv")</code>
Peek at the sheet	<code>df.head()</code> · <code>df.info()</code> · <code>df.describe()</code>
Pick columns	<code>df[["name", "price"]]</code>
Filter rows	<code>df[df["price"] > 100]</code>
Pivot table	<code>df.groupby("city")["sales"].sum()</code>
VLOOKUP / SQL JOIN	<code>df.merge(other, on="id", how="left")</code>
Fix empty cells	<code>df.dropna()</code> or <code>df.fillna(0)</code>

Mental formula — every pandas job is L-E-T-S:

1. **Load** → `read_csv`
2. **Explore** → `head` / `info` / `describe`
3. **Transform** → `select`, `filter`, `groupby`, `merge`, `fillna`
4. **Save** → `to_csv` / `to_parquet`

Golden rule: `groupby` = pivot table, `merge` = VLOOKUP. You already know pandas — in Excel words.

6 pathlib

Analogy: GPS Navigator vs Paper Directions. `os.path` is gluing strings like hand-written directions ("then add a backslash... unless Mac..."). A Path object is a GPS: it knows where it is, what exists nearby, and works in every country (OS).

```
from pathlib import Path

data_dir = Path("data")           # create the GPS point
file = data_dir / "sales" / "jan.csv" # "/" joins roads – works on ALL OSes

file.exists()                     # is this place real?
file.suffix, file.stem             # ".csv", "jan"
data_dir.mkdir(exist_ok=True)      # build the folder, no error if exists
for f in data_dir.glob("*.csv"): ... # find all CSVs (radar scan)
text = file.read_text()            # read whole file, one line
```

Mental formula:

1. **WHERE?** → `Path(...)` then join with `/`
2. **EXISTS?** → `.exists()` / create with `.mkdir(exist_ok=True)`
3. **FIND many?** → `.glob("*.csv")`

Golden rule: Never glue path strings with `+`. Paths are objects, joined with `/`.

7 Logging

Analogy: News Reporting Operations Center. Only 3 actors: the **Reporter** (Logger) notices events, the **Courier** (Handler) delivers them somewhere, the **Editor** (Formatter) styles the printed text.

```
[ Event ] → [ Reporter ] → [ Courier ] → [ Newspaper ]
pipeline   Logger          Handler      pipeline.log
```

```
logger = logging.getLogger("pipeline")           # 1 WHO reports? hire Reporter
logger.setLevel(logging.INFO)                     # Reporter's rule: INFO or worse
logger.handlers = []                             # fire old couriers (no duplicates)
handler = logging.FileHandler("pipeline.log", mode="w") # 2 WHERE? hire Courier
fmt = logging.Formatter("%(asctime)s - %(levelname)s - %(message)s") # 3 HOW look?
handler.setFormatter(fmt)                         # 4 CONNECT: editor → courier
logger.addHandler(handler)                        # courier → reporter

logger.info("Pipeline started")                   # Reporter works!
logger.error("Failed to process row 42")
```

Level	Newsroom meaning
DEBUG	reporter's private notes
INFO	normal news ("pipeline started")
WARNING	storm warning — odd, but running
ERROR	front-page bad news — something failed
CRITICAL	presses on fire — program can't continue

Mental formula — 4 questions: 1. **WHO** reports? getLogger · 2. **WHERE** to? FileHandler · 3. **HOW** look? Formatter · 4. **CONNECT** them: setFormatter + addHandler

Golden rule: *Loggers report events. Handlers deliver them. Formatters style them. (And `print()` is a rumor, not journalism.)*

8 The Pipeline — All 8 Concepts in One Story

Analogy: A Restaurant Doing Dinner Service. One kitchen (venv), a printed menu with exact measurements (type hints), an order form (dataclass config), a GPS to the pantry (pathlib), a robot sous-chef (pandas/NumPy), and a reporter writing tonight's review (logging).

```
Config (dataclass) → find files (pathlib.glob) → load + clean (pandas)
    |
    └── every step narrated by the Reporter (logging) → pipeline.log
```

Mental formula — the pipeline recipe (order matters):

1. **CONFIGURE** → a `@dataclass` holds paths + settings
2. **DISCOVER** → `Path.glob("*.csv")` finds the input files
3. **LOAD + VALIDATE** → `pd.read_csv`, check/clean bad rows
4. **TRANSFORM** → vectorized math (NumPy) + groupby (pandas)
5. **REPORT** → `logger.info(...)` each step; save output + log file

Golden rule of Module 1: *Typed config in, discovered files through, clean data out — and the log tells the whole story.*